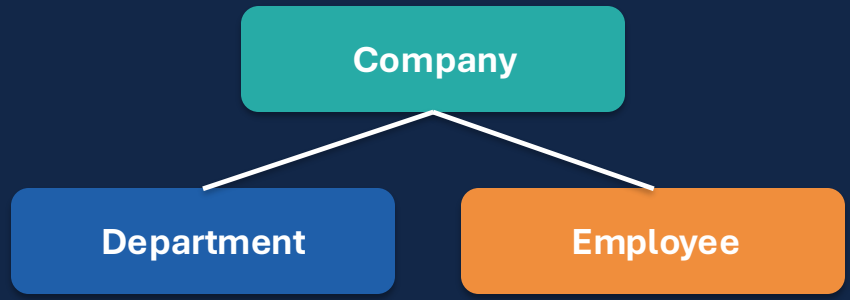
C2 Project Presentation

Composite Design Pattern

HR Management System: Modeling organizational hierarchies with clean, recursive structure



Structured summary from three Word files

Overview

Presentation Structure

The slides follow the exact order of the uploaded files:

01 Problem Analysis

Ayat file: Real-world hierarchy problem, main design challenges, and the trade-off analysis between Composite and Flyweight.

02 Pattern Justification

Noor file: Why Composite fits the HR system, how it removes conditionals, and which design principles it supports.

03 Pattern Application

Amna file: Component, Leaf, and Composite roles, including recursive delegation in calculateSalary().

FILE 1: AYAT

Problem Analysis & Trade-off Report

Understanding the software problem before selecting the pattern.

File 1: Ayat

Real-World Software Problem

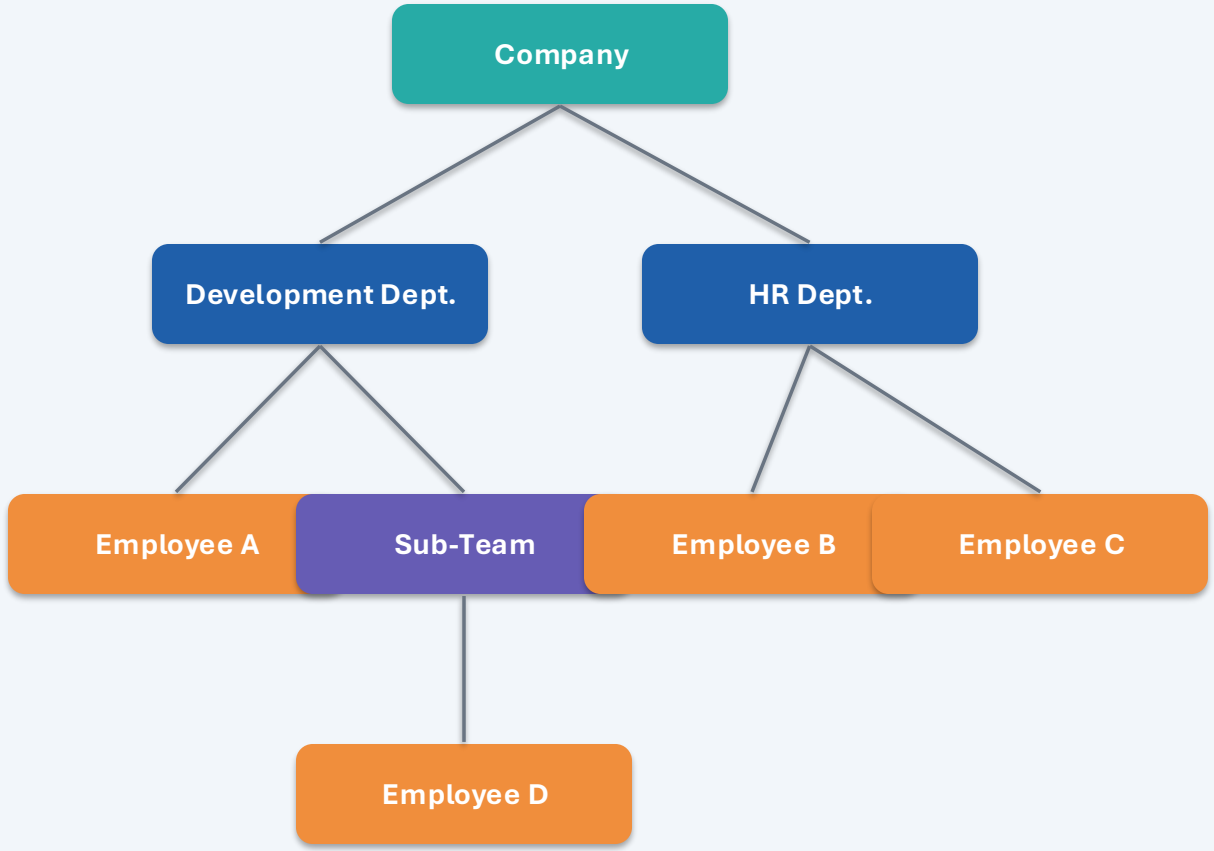
Enterprise HR systems are not flat lists.

An organization works like a recursive tree.

Departments and branches act as containers.

Employees act as individual leaf objects.

Budget or salary calculations must include nested teams.



Main issue: manual loops make the code rigid, repetitive, and difficult to maintain when the hierarchy grows.

File 1: Ayat

Main Design Challenges

1. Uniform Treatment

1

Client code should call calculateSalary() without checking whether the target is one Employee or a whole Department.

2. Recursive Aggregation

2

Each level must calculate its own data and combine it with the data of all its children.

3. Scalability & Depth

3

The hierarchy must support future levels such as regions, divisions, groups, and sub-departments.

The selected pattern must make growth easier, not more complicated.

File 1: Ayat

Design Pattern Trade-off Analysis

Pattern	Strength	Limitation for this problem
Flyweight	Optimizes memory when many objects share repeated data.	Does not represent containment or nesting, so it cannot model the company tree effectively.
Composite	Models part-whole tree structures and treats leaves and containers uniformly.	The shared interface can become slightly “fat,” because some methods may not fit every object perfectly.
Decision	**Composite directly solves the hierarchy problem.**	**The minor interface trade-off is acceptable because the structure becomes cleaner and easier to maintain.**

File 1: Ayat

Strategic Justification

Why Composite was the stronger choice

Structure First

The main goal was not only to save memory; it was to model a real organizational structure with nested levels.

Polymorphism

The system can handle Employees and Departments through the same OrganizationComponent abstraction.

Maintainability

Recursive structure reduces rigid manual loops and makes future hierarchy changes easier to support.

FILE 2: NOOR

Design Pattern Justification

Explaining why the Composite Pattern fits the HR Management System.

File 2: Noor

Selected Design Pattern: Composite

The Composite Pattern belongs to the Structural category and is used to treat individual objects and groups of objects uniformly.

Individual Object

Employee
A single unit with its own data and salary.

Composition of Objects

Department
A container that can hold employees and sub-departments.

Uniform Contract

OrganizationComponent
Shared operations: showDetails() and calculateSalary().

Result: the client can use the same commands without knowing the exact object type.

File 2: Noor

Why It Fits the HR System

Recursive Composition

Departments can contain Employees and other Departments because both implement OrganizationComponent.

Unified Execution

CompanyApp can call showDetails() or calculateSalary() on any component in the tree.

Cleaner Code

The pattern removes instanceof checks and long if-else blocks by using polymorphism.

Full Tree Calculation

Calling calculateSalary() at the company level triggers calculation across the whole hierarchy.

File 2: Noor

Advanced Design Principles Supported

OCP

Open/Closed Principle

New entities can be added by creating new classes that implement OrganizationComponent.

LSP

Liskov Substitution Principle

Employee and Department can be used wherever OrganizationComponent is expected.

DIP

Dependency Inversion Principle

High-level and low-level classes depend on a shared interface, not each other directly.

SRP

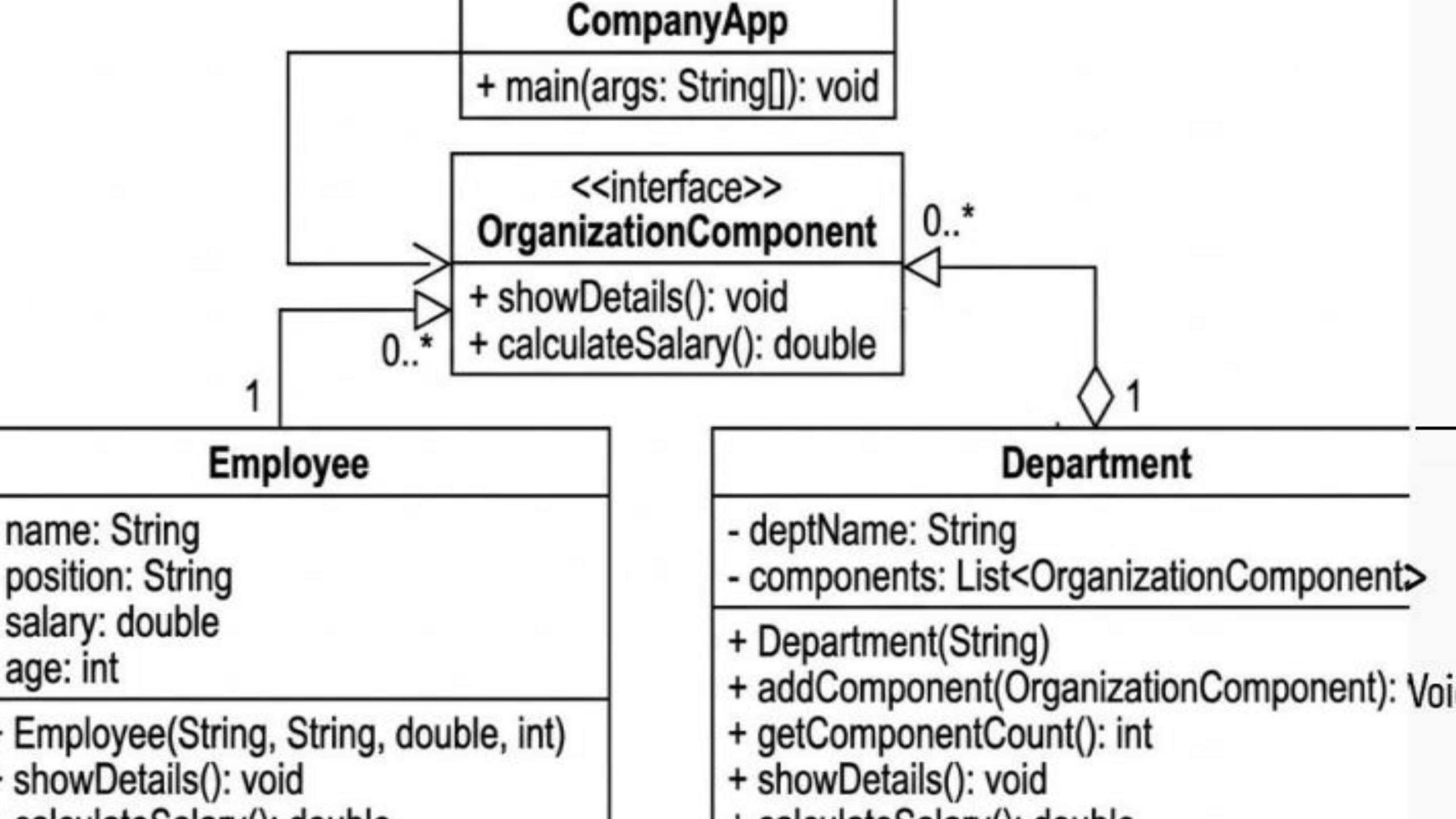
Single Responsibility Principle

Employee manages individual data; Department manages structure and aggregation.

FILE 3: AMNA

Applying the Pattern in the Solution

Connecting the design pattern roles to the project classes.


CompanyApp
+ main(args: String[]): void
<<interface>>
OrganizationComponent
+ showDetails(): void
+ calculateSalary(): double
0..*
0..*
1
1
Employee
name: String
position: String
salary: double
age: int
- Employee(String, String, double, int)
- showDetails(): void
Department
- deptName: String
- components: List<OrganizationComponent>
+ Department(String)
+ addComponent(OrganizationComponent): Voi
+ getComponentCount(): int
+ showDetails(): void

Conclusion

Final Conclusion

Composite is the most suitable solution because the HR system is naturally hierarchical.

Solves the Core Problem

It represents departments, branches, sub-teams, and employees as one recursive tree.

Improves Code Quality

It replaces complex type-checking with polymorphism and a shared interface.

Supports Future Growth

It allows the organization to add new structure levels without breaking the existing design.

This makes the system cleaner, easier to maintain, and scalable for real enterprise use.

Code Implementation

Code Slide 1: OrganizationComponent

Java implementation of the Composite Design Pattern in the HR Management System

```
public interface OrganizationComponent {
    void showDetails();
    double calculateSalary();
}
```

Component Interface

The base contract for the hierarchy. Both Employee and Department implement the same shared operations.

No previous slides were changed

Code Slide 2: Employee Class

Java implementation of the Composite Design Pattern in the HR Management System

```
public class Employee implements OrganizationComponent {
    private String name;
    private String position;
    private double salary;
    private int age;

    public Employee(String name, String position, double salary) {
        this.name = name;
        this.position = position;
        this.salary = salary;
    }

    @Override
    public void showDetails() {
        System.out.println(
            "   - Employee: " + name +
            " | Position: " + position +
            " | Salary: " + salary
        );
    }

    @Override
    public double calculateSalary() {
        return salary;
    }
}
```

Leaf Class

Represents one employee. It has no children, so calculateSalary() simply returns the employee salary.

No previous slides were changed

Code Implementation

Code Slide 3: Department Class

Code is split into two panels to keep the full class readable on one slide

Composite Class

```
import java.util.ArrayList;
import java.util.List;

public class Department implements OrganizationComponent {
    private String deptName;
    private List<OrganizationComponent> components = new ArrayList<>();

    public Department(String name) {
        this.deptName = name;
    }

    public void addComponent(OrganizationComponent component) {
        components.add(component);
    }

    @Override
    public void showDetails() {
        System.out.println("\n--- Department: " + deptName + " ---");
        for (OrganizationComponent component : components) {
            component.showDetails();
        }
    }
```

```
    @Override
    public double calculateSalary() {
        double totalSalary = 0;

        for (OrganizationComponent component : components) {
            totalSalary += component.calculateSalary();
        }
        return totalSalary;
    }

    public int getComponentCount() {
        return components.size();
    }
}
```

No previous slides were changed

Code Implementation

Code Slide 4: CompanyApp Main Class

Code is split into two panels to keep the full class readable on one slide

Client / Main Class

```
public class CompanyApp {
    public static void main(String[] args) {
        // Create individual employees
        Employee emp1 = new Employee("Ahmed", "Software Developer", 5000);
        Employee emp2 = new Employee("Sarah", "Backend Developer", 6000);
        Employee emp3 = new Employee("Khaled", "UI/UX Designer", 7000);

        // Create departments
        Department developmentDept = new Department("Development");
        Department designDept = new Department("Design");
        Department company = new Department("Main Company");

        // Build the hierarchical tree structure
        developmentDept.addComponent(emp1);
        developmentDept.addComponent(emp2);
        designDept.addComponent(emp3);
```

```
        // Add departments to the main company
        company.addComponent(developmentDept);
        company.addComponent(designDept);

        // Run operations
        System.out.println("--- Full Company Structure ---");
        company.showDetails();

        System.out.println("\n--- Total Financial Cost ---");
        System.out.println(
            "Total Company Salaries: " + company.calculateSalary()
        );
    }
}
```

No previous slides were changed